

Mise en place de CI/CD avec GitLab pour un projet Node.js

Table des Matières

1. Introduction
2. Installation de GitLab et GitLab Runner
3. Configuration du Projet Node.js
4. Création du Pipeline CI/CD
 - Build
 - Test
 - Déploiement sur Docker
5. Conclusion

1. Introduction

Cette documentation explique comment mettre en place un pipeline CI/CD avec GitLab pour un projet Node.js. L'objectif est d'automatiser le processus de build, test, et déploiement de l'application.

2. Installation de GitLab et GitLab Runner

Prérequis:

- VM ubuntu 22.04 LTS sur Virtual Box
- Docker
- Docker Compose

Docker Compose pour GitLab et GitLab Runner:

- Créez un dossier nommé « gitlab »
- Créez un fichier `docker-compose.yml` avec le contenu suivant:

```
version: "3.3"
services:
  gitlab:
    image: 'gitlab/gitlab-ce:latest'
    restart: always
    hostname: 'gitlab.driss.com'
    environment:
      GITLAB_OMNIBUS_CONFIG: |
        external_url 'http://gitlab.driss.com'
    ports:
      - '80:80'
      - '2222:2222'
    volumes:
      - '${PWD}/config:/etc/gitlab'
      - '${PWD}/logs:/var/log/gitlab'
      - '${PWD}/data:/var/opt/gitlab'
    networks:
      - gitlab
  gitlab-runner:
    image: 'gitlab/gitlab-runner:latest'
    depends_on:
      - gitlab
    environment:
      GIT_SSL_NO_VERIFY: "1"
    volumes:
      - '/var/run/docker.sock:/var/run/docker.sock'
      - './config-runner:/etc/gitlab-runner'
    ports:
      - '8093:8093'
    restart: unless-stopped
    networks:
      - gitlab
networks:
  gitlab:
```

Démarrer les services:

```
docker-compose up -d
```

3. Configuration du Projet Node.js

Création d'un Projet dans GitLab

1. Connectez-vous à l'interface Web de GitLab.

- **Username** : root

- **password** : utiliser initial root password. (/etc/gitlab/initial_root_password)

2. Cliquez sur le bouton 'Nouveau projet' ou 'New Project'.
3. Sélectionnez 'Créer un projet vide' ou 'Create blank project'.
4. Donnez un nom à votre projet, par exemple 'nodeapp'.
5. Cliquez sur 'Créer un projet' ou 'Create project'.

Enregistrement du Runner Docker

Pour enregistrer le GitLab Runner avec Docker, suivez ces étapes :

1. Exécutez la commande suivante pour enregistrer le runner :

```
docker run --rm -it -v $(pwd)/runner-config:/etc/gitlab-runner  
gitlab/gitlab-runner:latest register
```

2. Vous serez invité à entrer les informations suivantes :
 - URL du GitLab (http://votre-ip-gitlab/)
 - Token du runner (trouvé dans l'interface de GitLab sous CI/CD -> Runners)
 - Description du runner
 - Tags du runner (séparés par des virgules)
 - Type d'exécuteur (choisissez 'docker')
3. Configurez le runner pour utiliser l'image Docker docker:latest.

Contenu du fichier config.toml :

```
concurrent = 1  
  
check_interval = 0  
  
connection_max_age = "15m0s"  
  
shutdown_timeout = 0  
  
[session_server]  
  
session_timeout = 1800
```

```
[[runners]]  
  name = "runderdind"  
  url = "http://192.168.148.118/"  
  id = 2  
  token = "9Bk5T8MySG4G8fyq1gM3"  
  token_obtained_at = 2024-06-26T01:40:50Z  
  token_expires_at = 0001-01-01T00:00:00Z  
  executor = "docker"  
  clone_url = "http://192.168.148.118/"  
  [runners.custom_build_dir]  
  [runners.cache]  
    MaxUploadedArchiveSize = 0  
    [runners.cache.s3]  
    [runners.cache.gcs]  
    [runners.cache.azure]  
  [runners.docker]  
    tls_verify = false  
    image = "docker:24.0.5"  
    privileged = true  
    disable_entrypoint_overwrite = false  
    oom_kill_disable = false  
    disable_cache = false
```

```
volumes = ["/cache"]
```

```
shm_size = 0
```

```
network_mtu = 0
```

Ajout d'une clé SSH à GitLab

1. Générez une clé SSH sur votre machine si ce n'est pas déjà fait:

```
ssh-keygen -t rsa -b 4096 -C "your_email@example.com"
```

2. Ajoutez votre clé SSH à l'agent SSH:

```
ssh-add ~/.ssh/id_rsa
```

3. Copiez le contenu de votre clé publique:

```
cat ~/.ssh/id_rsa.pub
```

4. Allez dans les 'Paramètres' ou 'Settings' de votre compte GitLab.

5. Cliquez sur 'Clés SSH' ou 'SSH Keys'.

6. Collez le contenu de votre clé publique dans le champ 'Key'.

7. Cliquez sur 'Ajouter une clé' ou 'Add key'.

Code source du projet 'nodeapp' :

- Clone le projet sur votre machine :

Utiliser la commande : `git clone https://gitlab.com/drissRiane/nodeapp.git nodeapp`

Ouvrir le dossier nodeapp avec vs code

Structure du projet :

...

Nom	Type
.git	Folder
node_modules	Folder
test	Folder
.gitlab-ci.yml	File
app.js	File
dockerfile	File
package.json	File
package-lock.json	File
README.md	File
test.js	File

Contenu du fichier `Dockerfile` :

```
FROM node:14

WORKDIR /usr/src/app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["node", "app.js"]
```

4. Création du Pipeline CI/CD

Allez dans le fichier `.gitlab-ci.yml` :

```
image: docker:24.0.5
cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths:
    - node_modules/
variables:
  DOCKER_DRIVER: overlay2
  DOCKER_TLS_CERTDIR: ""
  DOCKER_REGISTRY: 192.168.148.118:5000
```

```
DOCKER_IMAGE: $DOCKER_REGISTRY/myapp:$CI_COMMIT_SHA
services:
  - name: docker:24.0.5-dind
    command:
      - "--insecure-registry"
      - "192.168.148.118:5000"

stages:
  - build
  - test
  - deploy
build_job:
  stage: build
  script:
    - docker build -t $DOCKER_IMAGE .
    - docker push $DOCKER_IMAGE
before_script:
  - chmod +x node_modules/.bin/mocha
test_job:
  stage: test
  script:
    - docker run --rm $DOCKER_IMAGE npm test
deploy_to_Docker:
  stage: deploy
  script:
    - docker pull $DOCKER_IMAGE
    - docker run -d -p 3000:3000 $DOCKER_IMAGE
```

5. Conclusion

En suivant ce guide, vous avez mis en place un pipeline CI/CD complet pour un projet Node.js en utilisant GitLab CI/CD. Vous avez appris à automatiser les processus de build, test, déploiement, améliorant ainsi l'efficacité et la fiabilité de vos livraisons de code.